Course Code	Course Title	L	T	P	C
MACSE517	**Computer Architecture and Organization**	**3**	**0**	**2**	**4**
Pre-requisite	Nil	**Syllabus Version**			
		1			

Course Objectives

1. To provide knowledge on the basics of computer architectures and organization that lays the foundation to study high-performance architectures.
2. To impart skills to design and develop parallel programs using parallel computing platforms such as OpenMP, CUDA.
3. To familiarize on the domain-specific architectures and real-world implementations in modern processors.

Course Outcomes

1. Outline the developments in the evolution of computer architectures and parallel programming paradigms.
2. Design and evaluate memory hierarchies including multi-level cache systems and virtual memory.
3. Develop parallel programs using OpenMP and analyze performance parameters such as speed-up, and efficiency for parallel programs against serial programs.
4. Analyze GPU and SIMD architectures and implement data-level parallelism using CUDA.
5. Compare domain-specific architectures and evaluate their impact in modern data center applications.

Module:1	**Fundamentals of Computer Architecture**	**5 hours**

Introduction to Parallelism-Flynn's Classification: SISD, SIMD, MISD, MIMD-Pipelining- Instruction and Arithmetic Pipelines, Superscalar and Super Pipelined Architectures- Metrics for Performance Measurement.

Module:2	**Memory Hierarchy Design**	**8 hours**

Optimizations of Cache Performance - Memory Technology and Optimizations-Virtual Memory and Virtual Machines -The Design of Memory Hierarchy-Case Study: Memory Hierarchies in Intel Core i7 and ARM Cortex-A8.

Module:3	**Instruction-Level and Thread-Level Parallelism**	**11 hours**

Instruction-level Parallelism: Concepts and Challenges- Basic Compiler Techniques for Exposing ILP –Dynamic Scheduling-Limitations of ILP-Multithreading: Exploiting Thread-Level Parallelism –Shared Memory Multicore Systems, Performance Metrics for Shared-Memory Multicore Systems-Multithreaded Programming using OpenMP.

Module:4	**Data-Level Parallelism in Vector, SIMD, and GPU Architectures**	**10 hours**

Introduction, Vector Architecture, SIMD Instruction Set Extensions for Multimedia, Graphics Processing Units, GPU Memory Hierarchy, Detecting and Enhancing Loop- Level Parallelism, CUDA Programming, Case Study: Nvidia Maxwell.		
Module:5	**Domain-Specific Architectures**	**9 hours**
Introduction - Guidelines for DSAs - Google's Tensor Processing Unit: An Inference Data Center Accelerator - Microsoft Catapult: a Flexible Data Center Accelerator - Intel Crest: a Data Center Accelerator for Training.		
Module:6	**Contemporary Issues**	**2 hours**
	Total Lecture Hours:	**45 hours**

Text Book(s)

J.L. Hennessy and D.A. Patterson. , "**Computer Architecture: A Quantitative Approach.**", Morgan Kauffmann Publishers, 6th Edition, **2019**
Gerassimos Barlas, "**Multicore and GPU Programming: An Integrated Approach.**", Morgan Kaufmann, 2nd Edition, **2022**

Reference Books

William Stallings., "**Computer Organization and Architecture: Designing for Performance.**", Pearson, 11th Edition, **2022**
T. Deakin and T.G. Mattson. , "**Programming Your GPU with OpenMP: Performance Portability for GPUs.**", MIT Press Cambridge, 1st Edition, **2023**
J. Sanders and E. Kandrot, " **CUDA by Example: An Introduction to General-Purpose GPU Programming.**", Addison-Wesley Boston, 1st Edition, **2010**

Indicative Experiments

1. Create a new project using Visual Studio or an equivalent IDE that supports OpenMP. Configure the project properties to enable OpenMP support, compile, and execute the basic "hello world" program.	2 hours
2. Construct an OpenMP program using the parallel construct that prints the thread number from each thread in the parallel region to demonstrate how multiple threads execute concurrently.	2 hours
3. Develop a OpenMP program that calculates the execution time of a computational task using timing function and performance counters. Compare their properties.	2 hours
4. Implement an OpenMP program using various environment routines to access processor run-time information that simulates a group of students (threads) solving a set of problems simultaneously. Each student reports their ID, total students participating, maximum possible students, and processor details.	2 hours
5. Write an OpenMP program that simulates operations on a deck of cards. The program should use the loop construct to shuffle the deck in	4 hours

parallel, the sections construct to deal cards concurrently to two players, and the single construct to print a game start announcement exactly once. Use these constructs to demonstrate how parallel tasks can be organized efficiently. Also, explain the purpose and behavior of each construct.	
6. Generate an OpenMP program to simulate a bakery model with 20 customer orders for varying preparation times. Use the static, dynamic, and guided scheduling clauses to assign orders to multiple threads. Observe and compare how each scheduling method affects workload distribution and overall execution time.	2 hours
7. Use Pin instrumentation and Cache grind profiling to analyze cache usage and identify where cache misses happen.	2 hours
8. Track branch predictions and mispredictions during execution and identify inefficient branching and improve performance.	2 hours
9. Develop a CUDA-platform setup on NVIDIA / Google Colab.	2 hours
10. Build a CUDA program in C/C++ environment to add two large arrays using unified memory, validate the result with a CPU implementation, and measure GPU kernel execution time using CUDA events.	2 hours
11. Write a CUDA program in C/C++ environment to reverse the elements of a large array using a single thread block, utilize shared memory for efficient access, include boundary checks, and measure the execution time using CUDA events.	2 hours
12. Implement a CUDA program in C/C++ environment to perform matrix addition and multiplication using shared memory optimization, validate the results with CPU computation, and compare execution times using CUDA event profiling.	2 hours
13. Hands-on Lab: CNN Training with TensorFlow on TPUs.	4 hours
Total Laboratory Hours:	**30 hours**
Mode of Evaluation : Continuous Assessment Test, Digital Assignment, Quiz, Final Assessment Test, Lab Continuous Assessment, Lab Final Assessment	
Recommended by Board of Studies :	23-05-2025
Approved by Academic Council : No. 78	12-06-2025